Cyber Lab Short Codes

1a Caesar Cipher

```
import java.util.*;

class Main {

   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      String s = sc.nextLine();

      int k = sc.nextInt();

      for (char c : s.toCharArray())

         System.out.print(Character.isLetter(c) ?

            (char)((Character.isUpperCase(c) ? 'A':'a') +

            (c-(Character.isUpperCase(c)?'A':'a')+k)%26) : c);

   }

}
```

1b Playfair Cipher

```
import java.util.*;

class Main {

   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      String key = sc.next().toUpperCase().replace("J","I");

      String s = sc.next().toUpperCase().replace("J","I");

      String a = "";

      for(char c : (key+"ABCDEFGHIKLMNOPQRSTUVWXYZ").toCharArray())

         if(Character.isLetter(c) && !a.contains(""+c)) a+=c;

      char[][] m = new char[5][5];

      for(int i=0;i<25;i++) m[i/5][i%5]=a.charAt(i);

      s=s.replaceAll("[^A-Z]","");

      for(int i=0;i<s.length();i+=2) {

         char x=s.charAt(i), y=i+1<s.length()?s.charAt(i+1):'X';

         if(x==y)y='X';

         int r1=0,c1=0,r2=0,c2=0;

         for(int r=0;r<5;r++)for(int c=0;c<5;c++){
```

```
            if(m[r][c]==x){r1=r;c1=c;}
            if(m[r][c]==y){r2=r;c2=c;}
        }
        if(r1==r2){c1=(c1+1)%5;c2=(c2+1)%5;}
        else if(c1==c2){r1=(r1+1)%5;r2=(r2+1)%5;}
        else {int t=c1;c1=c2;c2=t;}
        System.out.print(""+m[r1][c1]+m[r2][c2]);
      }
   }
}
```

1c Hill Cipher

```
import java.util.*;
class Main {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      String s=sc.next().toUpperCase();
      int a=3,b=3,c=2,d=5;
      if(s.length()%2==1)s+="X";
      for(int i=0;i<s.length();i+=2) {
         int x=s.charAt(i)-65, y=s.charAt(i+1)-65;
         System.out.print((char)((a*x+b*y)%26+65));
         System.out.print((char)((c*x+d*y)%26+65));
      }
   }
}
```

1d Vigenère Cipher

```
import java.util.*;
class Main {
   public static void main(String[] args) {
      Scanner sc=new Scanner(System.in);
      String s=sc.next().toUpperCase(), k=sc.next().toUpperCase();
```

```
        for(int i=0;i<s.length();i++)
            System.out.print((char)((s.charAt(i)-65+
                k.charAt(i%k.length())-65)%26+65));
    }
}
```

2a Rail Fence Cipher

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        String s=sc.nextLine();
        int r=sc.nextInt();
        char[][] a=new char[r][s.length()];
        int row=0,dir=1;
        for(int i=0;i<s.length();i++){
            a[row][i]=s.charAt(i);
            if(row==0)dir=1;
            if(row==r-1)dir=-1;
            row+=dir;
        }
        for(char[] x:a)for(char c:x)if(c!=0)System.out.print(c);
    }
}
```

2b Row and Column Transposition

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        String s=sc.next().toUpperCase();
        int n=sc.nextInt();
        while(s.length()%n!=0)s+="X";
```

```
        for(int c=0;c<n;c++)
            for(int i=c;i<s.length();i+=n)
                System.out.print(s.charAt(i));
    }
}
```

3 DES Algorithm

```
import javax.crypto.*;
import java.util.*;
class Main {
    public static void main(String[] args) throws Exception {
        Scanner sc=new Scanner(System.in);
        KeyGenerator kg=KeyGenerator.getInstance("DES");
        SecretKey k=kg.generateKey();
        Cipher c=Cipher.getInstance("DES");
        c.init(Cipher.ENCRYPT_MODE,k);
        byte[] e=c.doFinal(sc.nextLine().getBytes());
        System.out.println(Base64.getEncoder().encodeToString(e));
        c.init(Cipher.DECRYPT_MODE,k);
        System.out.println(new String(c.doFinal(e)));
    }
}
```

4 AES Algorithm

```
import javax.crypto.*;
import javax.crypto.spec.*;
import java.util.*;
class Main {
    public static void main(String[] args) throws Exception {
        Scanner sc=new Scanner(System.in);
        byte[] key=new byte[16];
        Arrays.fill(key,(byte)1);
        SecretKeySpec k=new SecretKeySpec(key,"AES");
```

```
        Cipher c=Cipher.getInstance("AES");
        c.init(Cipher.ENCRYPT_MODE,k);
        byte[] e=c.doFinal(sc.nextLine().getBytes());
        System.out.println(Base64.getEncoder().encodeToString(e));
        c.init(Cipher.DECRYPT_MODE,k);
        System.out.println(new String(c.doFinal(e)));
    }
}
```

5 RSA Algorithm

```
<!DOCTYPE html>
<html>
<body>
<input id="msg" value="Hello">
<button onclick="run()">Encrypt/Decrypt</button>
<p id="out"></p>
<script>
async function run() {
  const k = await crypto.subtle.generateKey({
    name:"RSA-OAEP",
    modulusLength:2048,
    publicExponent:new Uint8Array([1,0,1]),
    hash:"SHA-256"
  },true,["encrypt","decrypt"]);
  const m=new TextEncoder().encode(msg.value);
  const e=await crypto.subtle.encrypt(
    {name:"RSA-OAEP"},k.publicKey,m);
  const d=await crypto.subtle.decrypt(
    {name:"RSA-OAEP"},k.privateKey,e);
  out.innerText=new TextDecoder().decode(d);
}
</script>
```

```
</body>
</html>
```

6 Diffie–Hellman Key Exchange

```
import javax.crypto.*;
import java.security.*;
class Main {
   public static void main(String[] args) throws Exception {
      KeyPairGenerator g=KeyPairGenerator.getInstance("DH");
      g.initialize(512);
      KeyPair a=g.generateKeyPair(), b=g.generateKeyPair();
      KeyAgreement x=KeyAgreement.getInstance("DH");
      x.init(a.getPrivate());
      x.doPhase(b.getPublic(),true);
      KeyAgreement y=KeyAgreement.getInstance("DH");
      y.init(b.getPrivate());
      y.doPhase(a.getPublic(),true);
      System.out.println(Arrays.equals(
         x.generateSecret(),y.generateSecret()));
   }
}
```

7 SHA-1 Algorithm

```
import java.security.*;
import java.util.*;
class Main {
   public static void main(String[] args) throws Exception {
      Scanner sc=new Scanner(System.in);
      byte[] h=MessageDigest.getInstance("SHA-1")
         .digest(sc.nextLine().getBytes());
      for(byte b:h)System.out.printf("%02x",b);
   }
}
```

8 GnuPG

```
import java.io.*;

class Main {

    public static void main(String[] args) throws Exception {

        String[][] cmds={

            {"gpg","--symmetric","--cipher-algo","AES256","data.txt"},

            {"gpg","--decrypt","data.txt.gpg"},

            {"gpg","--detach-sign","data.txt"},

            {"gpg","--verify","data.txt.sig","data.txt"}

        };

        for(String[] c:cmds) {

            Process p=new ProcessBuilder(c).inheritIO().start();

            p.waitFor();

        }

    }

}
```